do280-4.18: Days remaining **89** | Lab hours used **4/80**

DO280-4.22: Days remaining **89** | Lab hours used **0/80**

Red Hat OpenShift Administration II: Configuring a Production Cluster

Course | Lab Environment

Lab: Configure Network Security

Configure firewall rules to protect microservice communication, and also configure TLS encryption between those microservices and for external access.

Outcomes

- Encrypt internal traffic between pods by using TLS service secrets that OpenShift generates.
- Route external traffic to terminate TLS within the cluster.
- Restrict ingress traffic for a group of pods by using network policies.

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start network-review
```



Instructions

The `lab` command prepares the necessary files for this exercise in the `~/DO280/labs/network-review` directory. The command also deploys to the `network-review` project an API that is composed of a `product` microservice and a `stock` microservice.

The `product` microservice is the entry point to the API. The `stock` microservice provides only additional information to the `product` response. If the `product` microservice cannot reach the `stock` microservice, then the `product` microservice returns the `-1` value.

The developer deployed the API without the security configuration. You must configure TLS for end-to-end communications and restrict the ingress to pods for both microservices.

To complete the exercise, the following URLs must respond without errors:

- `https://stock.network-review.svc.cluster.local/product/1`
- `https://product.apps.lab.example.com/products`

The API URL of your OpenShift cluster is `https://api.lab.example.com:6443`, and the `oc` command is already installed on your `workstation` machine. You can log in to the cluster as the `admin` user with `redhatocp` as the password.

1. Create the `stock-service-cert` secret for the OpenShift service certificate in the `network-review` namespace to encrypt communications between the `product` and the `stock` microservices. Use the `~/DO280/labs/network-review/stock-service.yaml` file to configure the stock service with the `service.beta.openshift.io/serving-cert-secret-name: stock-service-cert` annotation.

   1.1. Use the `oc login` command to log in to your OpenShift cluster as the `admin` user.

   ```
   [student@workstation ~]$ oc login -u admin -p redhatocp \
   https://api.lab.example.com:6443
   Login successful.

   ...output omitted...
   ```

   1.2. Change to the `network-review` project.

   ```
   [student@workstation ~]$ oc project network-review
   Now using project "network-review" on server "https://api.lab.example.
   com:6443"
   ```

   1.3. Change to the `~/DO280/labs/network-review` directory to access the lab files.

   ```
   [student@workstation ~]$ cd ~/DO280/labs/network-review
   ```

   1.4. Edit the `stock-service.yaml` manifest to configure the `stock` service with the `service.beta.openshift.io/serving-cert-secret-name: stock-service-cert` annotation. This annotation creates the `stock-service-cert`

secret with the service certificate and the key.

```
apiVersion: v1
kind: Service
metadata:
  name: stock
  namespace: network-review
  annotations:
    service.beta.openshift.io/serving-cert-secret-name: stock-service-
cert
spec:
...output omitted...
```

1.5. Apply the `stock` service changes.

```
[student@workstation network-review]$ oc apply -f stock-service.yaml
service/stock configured
```

1.6. Verify that the `stock-service-cert` secret contains a valid certificate for the `stock.network-review.svc` hostname in the `tls.crt` secret key. Decode the secret output by using the `base64` command with the `-d` option. Then, use the `openssl x509` command to read the output from standard input, and use the `-text` option to print the certificate in text form.

```
[student@workstation network-review]$ oc get secret stock-service-cert
\
--output="jsonpath={.data.tls\.crt}" \
| base64 -d | openssl x509 -text
...output omitted...
      X509v3 Subject Alternative Name:
          DNS:stock.network-review.svc, DNS:stock.network-review.svc.c
luster.local
...output omitted...
```

Hide Solution

2. Configure TLS on the `stock` microservice by using the `stock-service-cert` secret that OpenShift generates.

   Use the following settings in the `~/DO280/labs/network-review/stock-deployment.yaml` file to configure TLS:

   - Set the path for the certificate and key to `/etc/pki/stock/`.
   - Set the `TLS_ENABLED` environment variable to `"true"`.
   - Update the liveness and readiness probes to use TLS.

   Edit the `~/DO280/labs/network-review/stock-service.yaml` file to update the `stock` service to listen on the standard HTTPS 443 port, and apply the updated file.

2.1. Edit the `stock` deployment in the `stock-deployment.yaml` file to enable TLS for the application and for the liveness and readiness probes.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: stock
  namespace: network-review
spec:
...output omitted...
    spec:
      containers:
        - name: stock
...output omitted...
          ports:
            - containerPort: 8085
          readinessProbe:
            httpGet:
              port: 8085
              path: /readyz
              scheme: HTTPS (1)
          livenessProbe:
            httpGet:
              port: 8085
              path: /livez
              scheme: HTTPS (2)
          env:
            - name: TLS_ENABLED
              value: "true" (3)
...output omitted...
```

(1) The scheme is set to `HTTPS`.

(2)

(3) The value is set to `true`.

2.2. Edit the `stock-deployment.yaml` file to mount the `stock-service-cert` secret in the `/etc/pki/stock/` path.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: stock
  namespace: network-review
spec:
...output omitted...
    spec:
      containers:
        - name: stock
...output omitted...
          env:
            - name: TLS_ENABLED
              value: "true"
          volumeMounts:
            - name: stock-service-cert
              mountPath: /etc/pki/stock/
      volumes:
        - name: stock-service-cert
          secret:
            defaultMode: 420
            secretName: stock-service-cert
```

2.3. Apply the `stock` deployment updates by using the `oc apply` command.

```
[student@workstation network-review]$ oc apply -f stock-deployment.ya
ml
deployment/stock configured
```

2.4. Edit the `stock-service.yaml` file to configure the `stock` service to listen on the standard HTTPS `443` port.

```
apiVersion: v1
kind: Service
metadata:
  name: stock
  namespace: network-review
  annotations:
    service.beta.openshift.io/serving-cert-secret-name: stock-service
-cert
spec:
  selector:
    app: stock
  ports:
    - port: 443
      targetPort: 8085
      name: https
```

2.5. Apply the `stock` service changes.

```
[student@workstation network-review]$ oc apply -f stock-service.yaml
service/stock configured
```

Hide Solution

3. Configure TLS between the `product` and the `stock` microservices by using the internal Certificate Authority (CA) from OpenShift. Verify that you can query the `stock` microservice by using HTTPS from the `product` microservice.

   Use the `~/DO280/labs/network-review/service-ca-configmap.yaml` file to add the `service.beta.openshift.io/inject-cabundle: "true"` annotation.

   Use the `~/DO280/labs/network-review/product-deployment.yaml` file to configure the `product` microservice with the following settings:

   - The `CERT_CA` environment variable that is set to `/etc/pki/ca-trust/extracted/pem/tls-ca-bundle.pem` to access the OpenShift CA.
   - The `STOCK_URL` environment variable is set to `https://stock.network-review.svc`

   3.1. Edit the configuration map in the `service-ca-configmap.yaml` file to add the `service.beta.openshift.io/inject-cabundle: "true"` annotation. This annotation injects the OpenShift internal CA into the `service-ca` configuration map.

   ```
   apiVersion: v1
   kind: ConfigMap
   metadata:
     name: service-ca
     namespace: network-review
     annotations:
       service.beta.openshift.io/inject-cabundle: "true"
   data: {}
   ```

   3.2. Create the `service-ca` configuration map.

   ```
   [student@workstation network-review]$ oc create -f service-ca-configmap.yaml
   configmap/service-ca created
   ```

   3.3. Verify that OpenShift injects the CA certificate by describing the `service-ca` configuration map.

```
[student@workstation network-review]$ oc describe configmap service-c
a
Name:         service-ca
Namespace:    network-review
Labels:       <none>
Annotations:  openshift.io/description:
                Configmap is added/updated with a data item containin
g the CA signing bundle that can be used to veirfy service-serving ce
rtificates
              openshift.io/owning-component: service-ca
              service.beta.openshift.io/inject-cabundle: true

Data
====
service-ca.crt:
-----
-----BEGIN CERTIFICATE-----
...output omitted...
```

3.4. Edit the `product-deployment.yaml` file to configure the `product` deployment to use the `service-ca` configuration map, to add the `CERT_CA` environment variable, and to update the `STOCK_URL` environment variable to use the HTTPS protocol.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: product
  namespace: network-review
spec:
...output omitted...
    spec:
      containers:
        - name: product
...output omitted...
          env:
            - name: CERT_CA
              value: "/etc/pki/ca-trust/extracted/pem/tls-ca-bundle.p
em"
            - name: TLS_ENABLED
              value: "false"
            - name: STOCK_URL
              value: "https://stock.network-review.svc"
          volumeMounts:
             - name: trusted-ca
               mountPath: /etc/pki/ca-trust/extracted/pem
      volumes:
        - name: trusted-ca
          configMap:
            defaultMode: 420
            name: service-ca
            items:
              - key: service-ca.crt
                path: tls-ca-bundle.pem
```

3.5. Apply the `product` deployment updates.

```
[student@workstation network-review]$ oc apply -f product-deployment.
yaml
deployment/product configured
```

3.6. Send a request to the `https://stock.network-review.svc/product/1` URL from `product` deployment to verify that you can query the `stock` microservice by using HTTPS. Run the `oc exec` command to run the `curl` command to send a request to the `stock` microservice.

```
[student@workstation network-review]$ oc exec deployment/product \
-- curl -sw '\n' https://stock.network-review.svc/product/1
10
```

NOTE

The `-w '\n'` option in the `curl` command prints a new line after the response.

Hide Solution

4. Configure TLS on the `product` microservice by using a signed certificate by a corporate CA to accept TLS connections from outside the cluster.

   You have the CA certificate and the signed certificate for the `product.apps.lab.example.com` domain in the `~/DO280/labs/network-review/certs` directory of the lab.

   Create a TLS secret named `passthrough-cert` by using the `~/DO280/labs/network-review/certs/product.pem` certificate, and the `~/DO280/labs/network-review/certs/product.key` key.

   Use the following settings in the deployment to configure TLS:

   - Set the path for the certificate and key to `/etc/pki/product/`.
   - Set the `TLS_ENABLED` environment variable to the `"true"` value.
   - Update the liveness and readiness probes to use TLS.

   4.1. Create the `passthrough-cert` secret by using the `product.pem` certificate and the `product.key` key from the lab directory.

   ```
   [student@workstation network-review]$ oc create secret tls passthroug
   h-cert \
   --cert certs/product.pem --key certs/product.key
   secret/passthrough-cert created
   ```

   4.2. Edit the `volumeMounts` section of the `product-deployment.yaml` file to mount the `passthrough-cert` secret on the `/etc/pki/product/` path.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: product
spec:
...output omitted...
    spec:
      containers:
        - name: product
...output omitted...
          volumeMounts:
            - name: passthrough-cert
              mountPath: /etc/pki/product/
            - name: trusted-ca
              mountPath: /etc/pki/ca-trust/extracted/pem
      volumes:
        - name: passthrough-cert
          secret:
            defaultMode: 420
            secretName: passthrough-cert
        - name: trusted-ca
          configMap:
            defaultMode: 420
            name: service-ca
            items:
              - key: service-ca.crt
                path: tls-ca-bundle.pem
```

4.3. Edit the `product-deployment.yaml` file to configure TLS for the application and for the liveness and readiness probes.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: product
spec:
...output omitted...
    spec:
      containers:
        - name: product
...output omitted...
          ports:
            - containerPort: 8080
          readinessProbe:
            httpGet:
              port: 8080
              path: /readyz
              scheme: HTTPS
          livenessProbe:
            httpGet:
              port: 8080
              path: /livez
              scheme: HTTPS
          env:
            - name: CERT_CA
              value: "/etc/pki/ca-trust/extracted/pem/tls-ca-bundle.p
em"
            - name: TLS_ENABLED
              value: "true"
            - name: STOCK_URL
              value: "https://stock.network-review.svc"
...output omitted...
```

4.4. Apply the `product` deployment updates by using the `oc apply` command.

```
[student@workstation network-review]$ oc apply -f product-deployment.
yaml
deployment.apps/product configured
```

Hide Solution

5. Expose the `product` microservice to outer cluster access by using the FQDN in the signed certificate by the corporate CA. Use the `product.apps.lab.example.com` hostname.

5.1. Create a passthrough route for the `product` service by using the `product.apps.lab.example.com` hostname.

```
[student@workstation network-review]$ oc create route passthrough pro
duct-https \
--service product --port 8080 \
--hostname product.apps.lab.example.com
route.route.openshift.io/product-https created
```

5.2. Verify that you can query the `product` microservice from outside the cluster by using the `curl` command with the `ca.pem` CA certificate.

```
[student@workstation network-review]$ curl -sw '\n' --cacert certs/c
a.pem \
https://product.apps.lab.example.com/products
[{"id":1,"name":"rpi4_4gb","stock":10},{"id":2,"name":"rpi4_8gb","sto
ck":5}]
```

Hide Solution

6. Configure network policies to accept only ingress connections to the `stock` pod on the `8085` port that come from a pod with the `app=product` label. Use the `~/DO280/labs/network-review/stock-ingresspolicy.yaml` file to configure the network policy.

6.1. Edit the `stock-ingresspolicy.yaml` to add the network policy specification.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: stock-ingress-policy
spec:
  podSelector:
    matchLabels:
      app: stock
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: product
      ports:
        - protocol: TCP
          port: 8085
```

6.2. Create the network policy.

```
[student@workstation network-review]$ oc create -f stock-ingresspolic
y.yaml
networkpolicy.networking.k8s.io/stock-ingress-policy created
```

Hide Solution

7. Configure network policies to accept only ingress connections to the `product` pod on the `8080` port that come from the OpenShift router pods. You can use the `~/DO280/labs/network-review/product-ingresspolicy.yaml` file to configure the network policy.

   7.1. Edit the `product-ingresspolicy.yaml` file to accept ingress connections from router pods by adding a namespace selector with the `policy-group.network.openshift.io/ingress` label.

   ```
   apiVersion: networking.k8s.io/v1
   kind: NetworkPolicy
   metadata:
     name: product-ingress-policy
   spec:
     podSelector:
       matchLabels:
         app: product
     ingress:
       - from:
           - namespaceSelector:
               matchLabels:
                 policy-group.network.openshift.io/ingress: ""
         ports:
           - protocol: TCP
             port: 8080
   ```

   7.2. Create the network policy.

   ```
   [student@workstation network-review]$ oc create -f product-ingresspol
   icy.yaml
   networkpolicy.networking.k8s.io/product-ingress-policy created
   ```

   7.3. Change to the home directory.

   ```
   [student@workstation network-review]$ cd
   ```

   Hide Solution

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade network-review
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish network-review
```

Revision: do280-4.22-ed1-20260827-78c07d2



Privacy Policy (https://www.redhat.com/en/about/privacy-policy?extIdCarryOver=true&sc_cid=701f2000001D8QoAAK)

Red Hat Training Policies (https://www.redhat.com/en/about/red-hat-training-policies)

Terms of Use (https://www.redhat.com/en/about/terms-use)

All policies and guidelines (https://www.redhat.com/en/about/all-policies-guidelines)

Release Notes (https://docs.redhat.com/en/documentation/red_hat_learning_subscription/1-latest/html-single/red_hat_learning_subscription_release_notes/index)

Cookie Preferences and Opt-Out Rights